



LAB REPORT- 02

Department of Computer Science & Engineering (CSE)

Faculty of Science and Information Technology (FSIT)

Spring 2026

Course Title : Operating Systems Lab
Course Code : CSE324
Course Credit : 1.5
Date of Submission : April 03 , 2026
Topic Covered : CPU Scheduling Algorithm
(FCFS & SJF)

Submitted By

Shafin Ahmed

232-15-184

65_A1

Department of Computer Science & Engineering

Submitted To

Husne Mubarak

Lecturer

Department of Computer Science & Engineering

Faculty of FSIT

Comment By Course Teacher:

--

Table of Contents

Experiment No.	Experiment Name	Page No.
01	FCFS Scheduling Algorithm (Non-Preemptive)	01-05
02	SJF Scheduling Algorithm (Non-Preemptive)	06-09

Experiment No: 01

Experiment Name: FCFS Scheduling Algorithm (Non-Preemptive)

Program Name: fcfs.c

Introduction

Overview

In this experiment, I have implemented the First Come First Serve (FCFS) scheduling algorithm using a C program. The primary objective of this experiment has been to understand how CPU scheduling works based on the arrival order of processes.

I have calculated the Finish Time (FT), Turnaround Time (TAT), and Waiting Time (WT) for each process. I have also computed the average Turnaround Time and Waiting Time. Finally, I have represented the execution using a Gantt Chart.

Topics Covered

- FCFS Scheduling Algorithm
 - Non-preemptive scheduling
 - Turnaround Time (TAT)
 - Waiting Time (WT)
 - Gantt Chart representation
 - Dynamic input handling
-

Environment / Platform

I have performed this experiment using:

- Programming Language: C
- IDE: CodeBlocks
- Operating System: Windows / Linux

Task

Description

The task was to implement the FCFS (First Come First Serve) scheduling algorithm where processes are executed in the order of their arrival time.

The program takes dynamic input:

- Number of processes
- Arrival Time
- Burst Time

The program then calculates:

- Finish Time (FT)
- Turnaround Time ($TAT = FT - AT$)
- Waiting Time ($WT = TAT - BT$)

It also calculates:

- Average Turnaround Time
- Average Waiting Time

Finally, the program displays a Gantt Chart.

Source Code

```
#include <stdio.h>

int main()
{
    int n, i, j;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n];
    float at[n], bt[n], ft[n], tat[n], wt[n];

    // Input nitesi
    for(i = 0; i < n; i++)
    {
        printf("\nProcess P%d\n", i + 1);
```

```

pid[i] = i + 1;

printf("Enter Arrival Time: ");
scanf("%f", &at[i]);

printf("Enter Burst Time: ");
scanf("%f", &bt[i]);
}

// FCFS rule: arrival time onujai sort kortesi
for(i = 0; i < n - 1; i++)
{
    for(j = i + 1; j < n; j++)
    {
        if(at[i] > at[j])
        {
            // AT swap
            float temp;
            temp = at[i];
            at[i] = at[j];
            at[j] = temp;

            // BT swap
            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;

            // PID swap
            int t;
            t = pid[i];
            pid[i] = pid[j];
            pid[j] = t;
        }
    }
}

// Finish Time calculate kortesi
ft[0] = at[0] + bt[0];

for(i = 1; i < n; i++)
{
    // jodi CPU idle thake (gap thake)
    if(ft[i-1] < at[i])
        ft[i] = at[i] + bt[i];
}

```

```

        else
            ft[i] = ft[i-1] + bt[i];
    }

    float total_tat = 0, total_wt = 0;

    // TAT & WT calculate
    for(i = 0; i < n; i++)
    {
        tat[i] = ft[i] - at[i];
        wt[i] = tat[i] - bt[i];

        total_tat += tat[i];
        total_wt += wt[i];
    }

    // Table print
    printf("\n\nProcess\tAT\tBT\tFT\tTAT\tWT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%.1f\t%.1f\t%.1f\t%.1f\t%.1f\n",
            pid[i], at[i], bt[i], ft[i], tat[i], wt[i]);
    }

    // Average
    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
    printf("\nAverage Waiting Time = %.2f", total_wt / n);

    // Gantt Chart print
    printf("\nGantt Chart:\n\n");

    for(i = 0; i < n; i++)
    {
        printf(" P%d |", pid[i]);
    }

    printf("\n");

    // Time line print
    printf("%.1f", at[0]);
    for(i = 0; i < n; i++)
    {
        printf("    %.1f", ft[i]);
    }

    printf("\n");

```

```

return 0;
}

```

Code Description (FCFS Scheduling Algorithm)

The given program implements the **First Come First Serve (FCFS)** CPU scheduling algorithm using C programming language. FCFS is one of the simplest non-preemptive scheduling algorithms where processes are executed in the order of their arrival times.

Working Principle

1. Input Phase

- The user inputs the number of processes.
- For each process, the program takes:
 - Arrival Time (AT)
 - Burst Time (BT)
- Each process is assigned a Process ID (PID).

2. Sorting

- Processes are sorted based on their **Arrival Time** using a simple sorting technique.
- This ensures that the process which arrives earlier is executed first.

3. Finish Time Calculation

- The first process's finish time is calculated as:
 - $FT[0] = AT[0] + BT[0]$
- For subsequent processes:
 - If the CPU is idle (i.e., previous finish time is less than the current arrival time):
 - $FT[i] = AT[i] + BT[i]$
 - Otherwise:
 - $FT[i] = FT[i-1] + BT[i]$

4. Turnaround Time and Waiting Time

- Turnaround Time (TAT) is calculated as:
 - $TAT = FT - AT$
- Waiting Time (WT) is calculated as:
 - $WT = TAT - BT$

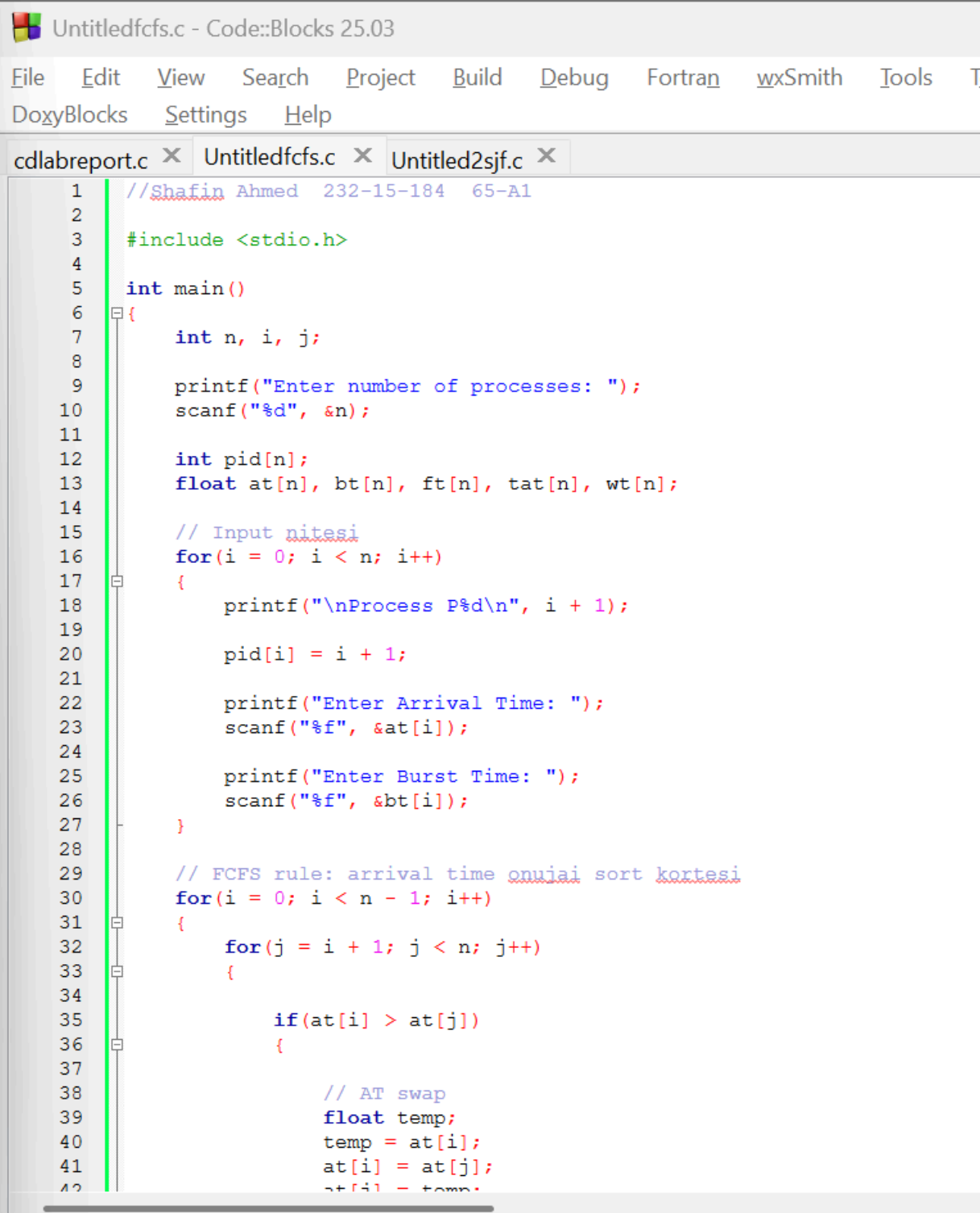
5. Averaging

- The program calculates:
 - Average Turnaround Time = Total TAT / Number of processes
 - Average Waiting Time = Total WT / Number of processes

6. Gantt Chart Generation

- The Gantt chart visually represents the execution order of processes.
- It shows:
 - Process execution sequence
 - Corresponding time intervals

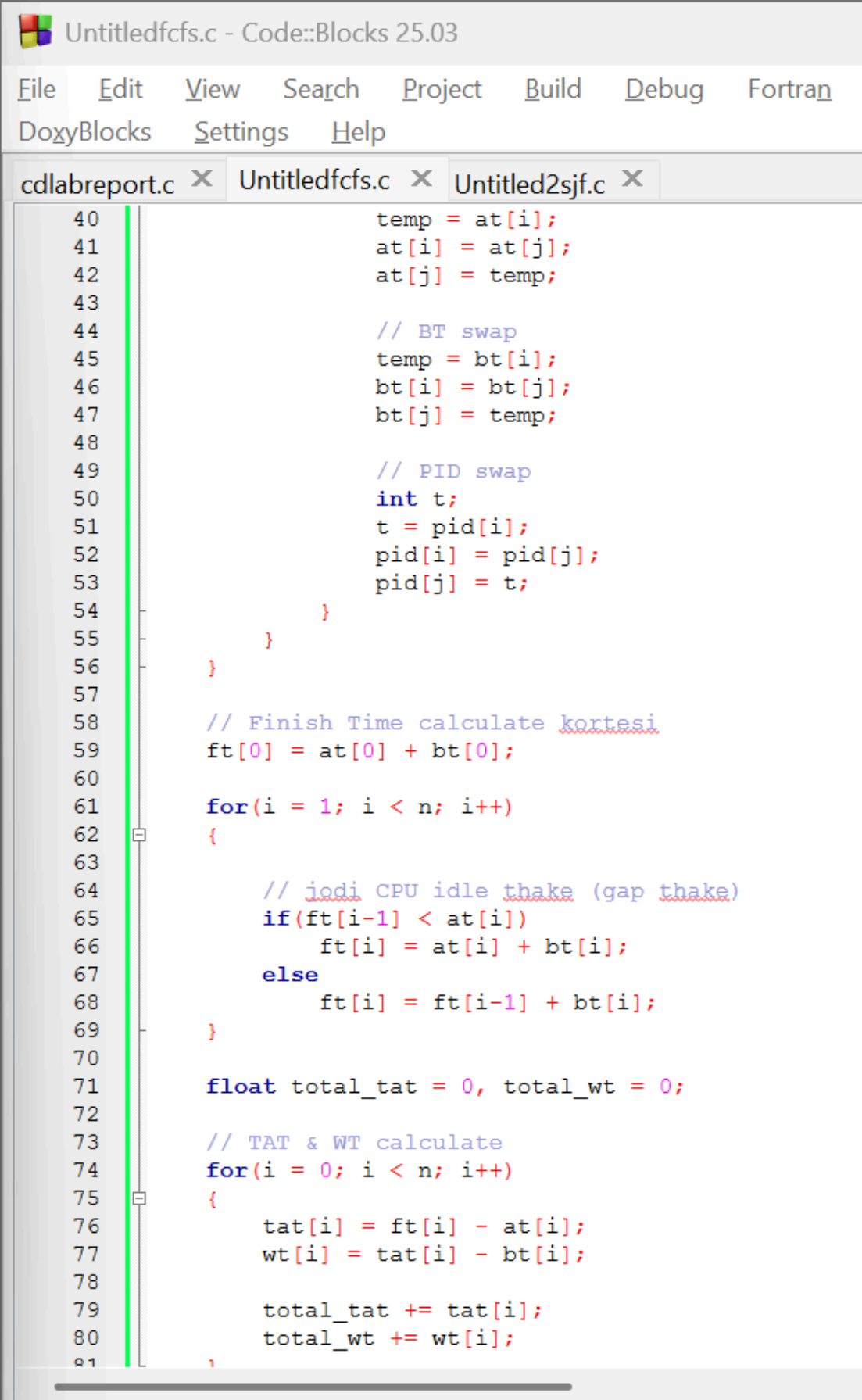
Input & Output



```

1 //Shafin Ahmed 232-15-184 65-A1
2
3 #include <stdio.h>
4
5 int main()
6 {
7     int n, i, j;
8
9     printf("Enter number of processes: ");
10    scanf("%d", &n);
11
12    int pid[n];
13    float at[n], bt[n], ft[n], tat[n], wt[n];
14
15    // Input nitesi
16    for(i = 0; i < n; i++)
17    {
18        printf("\nProcess P%d\n", i + 1);
19
20        pid[i] = i + 1;
21
22        printf("Enter Arrival Time: ");
23        scanf("%f", &at[i]);
24
25        printf("Enter Burst Time: ");
26        scanf("%f", &bt[i]);
27    }
28
29    // FCFS rule: arrival time onujai sort kortesi
30    for(i = 0; i < n - 1; i++)
31    {
32        for(j = i + 1; j < n; j++)
33        {
34
35            if(at[i] > at[j])
36            {
37
38                // AT swap
39                float temp;
40                temp = at[i];
41                at[i] = at[j];
42                at[j] = temp;

```

```

40         temp = at[i];
41         at[i] = at[j];
42         at[j] = temp;
43
44         // BT swap
45         temp = bt[i];
46         bt[i] = bt[j];
47         bt[j] = temp;
48
49         // PID swap
50         int t;
51         t = pid[i];
52         pid[i] = pid[j];
53         pid[j] = t;
54     }
55 }
56
57 // Finish Time calculate kortesi
58 ft[0] = at[0] + bt[0];
59
60 for(i = 1; i < n; i++)
61 {
62     // jodi CPU idle thake (gap thake)
63     if(ft[i-1] < at[i])
64         ft[i] = at[i] + bt[i];
65     else
66         ft[i] = ft[i-1] + bt[i];
67 }
68
69 float total_tat = 0, total_wt = 0;
70
71 // TAT & WT calculate
72 for(i = 0; i < n; i++)
73 {
74     tat[i] = ft[i] - at[i];
75     wt[i] = tat[i] - bt[i];
76
77     total_tat += tat[i];
78     total_wt += wt[i];
79 }
80
81

```

```

78
79     total_tat += tat[i];
80     total_wt += wt[i];
81 }
82
83 // Table print
84 printf("\n\nProcess\tAT\tBT\tFT\tTAT\tWT\n");
85
86 for(i = 0; i < n; i++)
87 {
88     printf("P%d\t%.1f\t%.1f\t%.1f\t%.1f\t%.1f\n",
89           pid[i], at[i], bt[i], ft[i], tat[i], wt[i]);
90 }
91
92 // Average
93 printf("\nAverage Turnaround Time = %.2f", total_tat / n);
94 printf("\nAverage Waiting Time = %.2f\n", total_wt / n);
95
96 // Gantt Chart print
97 printf("\nGantt Chart:\n\n|");
98
99 for(i = 0; i < n; i++)
100 {
101     printf("  P%d  |", pid[i]);
102 }
103
104 printf("\n");
105
106 // Time line print
107 printf("%.1f", at[0]);
108 for(i = 0; i < n; i++)
109 {
110     printf("      %.1f", ft[i]);
111 }
112
113 printf("\n");
114
115 return 0;
116 }
117
118

```

Figure 1: Source Code in CodeBlocks

```

C:\Users\admin\Documents\L × + - □ ×
Enter number of processes: 3

Process P1
Enter Arrival Time: 1
Enter Burst Time: 27

Process P2
Enter Arrival Time: 2
Enter Burst Time: 4.8

Process P3
Enter Arrival Time: 4
Enter Burst Time: 8.8

Process AT      BT      FT      TAT      WT
P1      1.0     27.0    28.0    27.0     0.0
P2      2.0      4.8    32.8    30.8    26.0
P3      4.0      8.8    41.6    37.6    28.8

Average Turnaround Time = 31.80
Average Waiting Time = 18.27

Gantt Chart:

| P1 | P2 | P3 |
1.0  28.0  32.8  41.6

Process returned 0 (0x0)  execution time : 42.957 s
Press any key to continue.

```

Figure 2: Program Execution Output

Discussion

Reflection

Through this experiment, I have learned how the FCFS scheduling algorithm works in a non-preemptive environment. I have understood that processes are executed strictly based on their arrival order.

I have also learned how to calculate Turnaround Time and Waiting Time, which are important performance metrics in operating systems.

Challenges Faced

Initially, I had to ensure correct sorting based on Arrival Time. I also carefully handled time calculation to avoid logical errors. After correcting these, the program produced accurate results.

Conclusion

In this experiment, I have successfully implemented the FCFS scheduling algorithm using C. I have calculated all required parameters and visualized execution using a Gantt Chart.

This concept is very important in operating systems for understanding CPU scheduling and process management.

Experiment No: 02

Experiment Name: SJF Scheduling Algorithm (Non-Preemptive)

Program Name: sjf.c

Introduction

Overview

In this experiment, I have implemented the Shortest Job First (SJF) scheduling algorithm in a non-preemptive manner using a C program. The primary objective of this experiment has been to understand how CPU scheduling can be optimized by selecting the process with the shortest burst time among the available processes.

I have calculated the Finish Time (FT), Turnaround Time (TAT), and Waiting Time (WT) for each process. I have also computed the average Turnaround Time and Waiting Time. Finally, I have represented the execution sequence using a Gantt Chart.

Topics Covered

- SJF Scheduling Algorithm
 - Non-preemptive scheduling
 - Turnaround Time (TAT)
 - Waiting Time (WT)
 - Gantt Chart representation
 - Dynamic process selection
-

Environment / Platform

I have performed this experiment using:

- Programming Language: C
- IDE: CodeBlocks
- Operating System: Windows / Linux

Task

Description

The task was to implement the Shortest Job First (SJF) scheduling algorithm where the process with the smallest burst time is selected from the available processes at any given time.

The program takes dynamic input:

- Number of processes
- Arrival Time
- Burst Time

The program then calculates:

- Finish Time (FT)
- Turnaround Time (TAT = FT - AT)
- Waiting Time (WT = TAT - BT)

It also calculates:

- Average Turnaround Time
- Average Waiting Time

Finally, the program displays a Gantt Chart showing execution order.

Source Code

```
#include <stdio.h>

int main()
{
    int n, i, completed = 0;
    float time = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], done[n];

    float at[n], bt[n], ft[n], tat[n], wt[n];

    // Input nitesi
```

```

for(i = 0; i < n; i++)
{
    printf("\nProcess P%d\n", i + 1);

    pid[i] = i + 1;

    printf("Enter Arrival Time: ");
    scanf("%f", &at[i]);

    printf("Enter Burst Time: ");
    scanf("%f", &bt[i]);

    done[i] = 0; // process ekhono complete hoy nai
}

float total_tat = 0, total_wt = 0;

// SJF Non-preemptive main logic
while(completed < n)
{
    int min_index = -1;

    // ready process der moddhe smallest BT khujtesi
    for(i = 0; i < n; i++)
    {
        if(at[i] <= time && done[i] == 0)
        {
            if(min_index == -1 || bt[i] < bt[min_index])
            {
                min_index = i;
            }
        }
    }

    // jodi kono process ready na thake
    if(min_index == -1)
    {
        time++; // CPU idle
    }
    else
    {
        // process execute kortesi
        ft[min_index] = time + bt[min_index];

        tat[min_index] = ft[min_index] - at[min_index];
        wt[min_index] = tat[min_index] - bt[min_index];

        total_tat += tat[min_index];
    }
}

```

```

        total_wt += wt[min_index];

        time = ft[min_index]; // time update

        done[min_index] = 1;
        completed++;
    }
}

// Output table
printf("\n\nProcess\tAT\tBT\tFT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%.1f\t%.1f\t%.1f\t%.1f\t%.1f\n",
        pid[i], at[i], bt[i], ft[i], tat[i], wt[i]);
}

// Average
printf("\nAverage Turnaround Time = %.2f", total_tat / n);
printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

// Gantt Chart generate kortesi
printf("\nGantt Chart:\n\n");

int temp_done[n];
for(i = 0; i < n; i++) temp_done[i] = 0;

float gantt_time[n+1];
int k = 0;

time = 0;
completed = 0;

// start time
gantt_time[k++] = time;

while(completed < n)
{
    int min_index = -1;

    for(i = 0; i < n; i++)
    {
        if(at[i] <= time && temp_done[i] == 0)
        {
            if(min_index == -1 || bt[i] < bt[min_index])
            {
                min_index = i;
            }
        }
    }
}

```



```

    }
}

if(min_index == -1)
{
    time++;
}
else
{
    printf(" P%d |", pid[min_index]);

    time += bt[min_index];
    gantt_time[k++] = time;

    temp_done[min_index] = 1;
    completed++;
}
}

printf("\n");

// timeline print
for(i = 0; i < k; i++)
{
    printf("%.1ft", gantt_time[i]);
}

printf("\n");

return 0;
}

```

Code Description

The given program implements the **Shortest Job First (SJF) Non-Preemptive** CPU scheduling algorithm using the C programming language. In this scheduling technique, the process with the smallest burst time is selected for execution among the available processes at a given time..

Working Principle

1. Input Phase

- The user inputs the number of processes.
- For each process:

- Arrival Time (AT)
 - Burst Time (BT)
 - Each process is assigned a unique Process ID (PID).
 - A `done[ ]` array is used to track whether a process has completed execution.
 - 2. **SJF Scheduling Logic**
 - The program uses a loop that continues until all processes are completed.
 - At each time unit:
 - The algorithm scans all processes that have arrived ($AT \leq \text{current time}$) and are not yet completed.
 - Among these, it selects the process with the **minimum burst time**.
 - If no process is available at the current time, the CPU remains idle and time is incremented.
 - 3. **Execution of Process**
 - Once the shortest job is selected:
 - Finish Time is calculated as:
 - $FT = \text{current time} + BT$
 - Turnaround Time:
 - $TAT = FT - AT$
 - Waiting Time:
 - $WT = TAT - BT$
 - The current time is updated to the finish time of the executed process.
 - The process is marked as completed.
 - 4. **Repetition**
 - The above steps repeat until all processes are completed.
 - 5. **Gantt Chart Generation**
 - A second traversal is performed to generate the Gantt chart.
 - It prints the order in which processes are executed.
 - A timeline array stores the start and finish times of each executed process.
-

Input & Output

```

1 // Shafin Ahmed 232-15-184 65-A1
2
3 #include <stdio.h>
4
5 int main()
6 {
7     int n, i, completed = 0;
8     float time = 0;
9
10    printf("Enter number of processes: ");
11    scanf("%d", &n);
12
13    int pid[n], done[n];
14
15    float at[n], bt[n], ft[n], tat[n], wt[n];
16
17    // Input nitesi
18    for(i = 0; i < n; i++)
19    {
20        printf("\nProcess P%d\n", i + 1);
21
22        pid[i] = i + 1;
23
24        printf("Enter Arrival Time: ");
25        scanf("%f", &at[i]);
26
27        printf("Enter Burst Time: ");
28        scanf("%f", &bt[i]);
29
30        done[i] = 0; // process ekhono complete hoy nai
31    }
32
33    float total_tat = 0, total_wt = 0;
34
35    // SJF Non-preemptive main logic
36    while(completed < n)
37    {
38        int min_index = -1;
39
40        // ready process der moddhe smallest BT khuitesi
41        for(i = 0; i < n; i++)
42        {
43            if(at[i] <= time && done[i] == 0)
44            {
45                if(min_index == -1 || bt[i] < bt[min_index])
46                {
47                    min_index = i;
48                }
49            }
50        }
51
52        // Process min_index is selected
53        if(min_index != -1)
54        {
55            completed++;
56            time += bt[min_index];
57            ft[min_index] = time;
58            tat[min_index] = ft[min_index] - at[min_index];
59            wt[min_index] = ft[min_index] - bt[min_index];
60            done[min_index] = 1;
61        }
62
63        total_tat += tat[min_index];
64        total_wt += wt[min_index];
65    }
66
67    printf("Average TAT: %.2f\n", total_tat / n);
68    printf("Average WT: %.2f\n", total_wt / n);
69    printf("Average FT: %.2f\n", total_tat / n + total_wt / n);
70
71    return 0;
72 }

```

Untitled2sjf.c - Code::Blocks 25.03

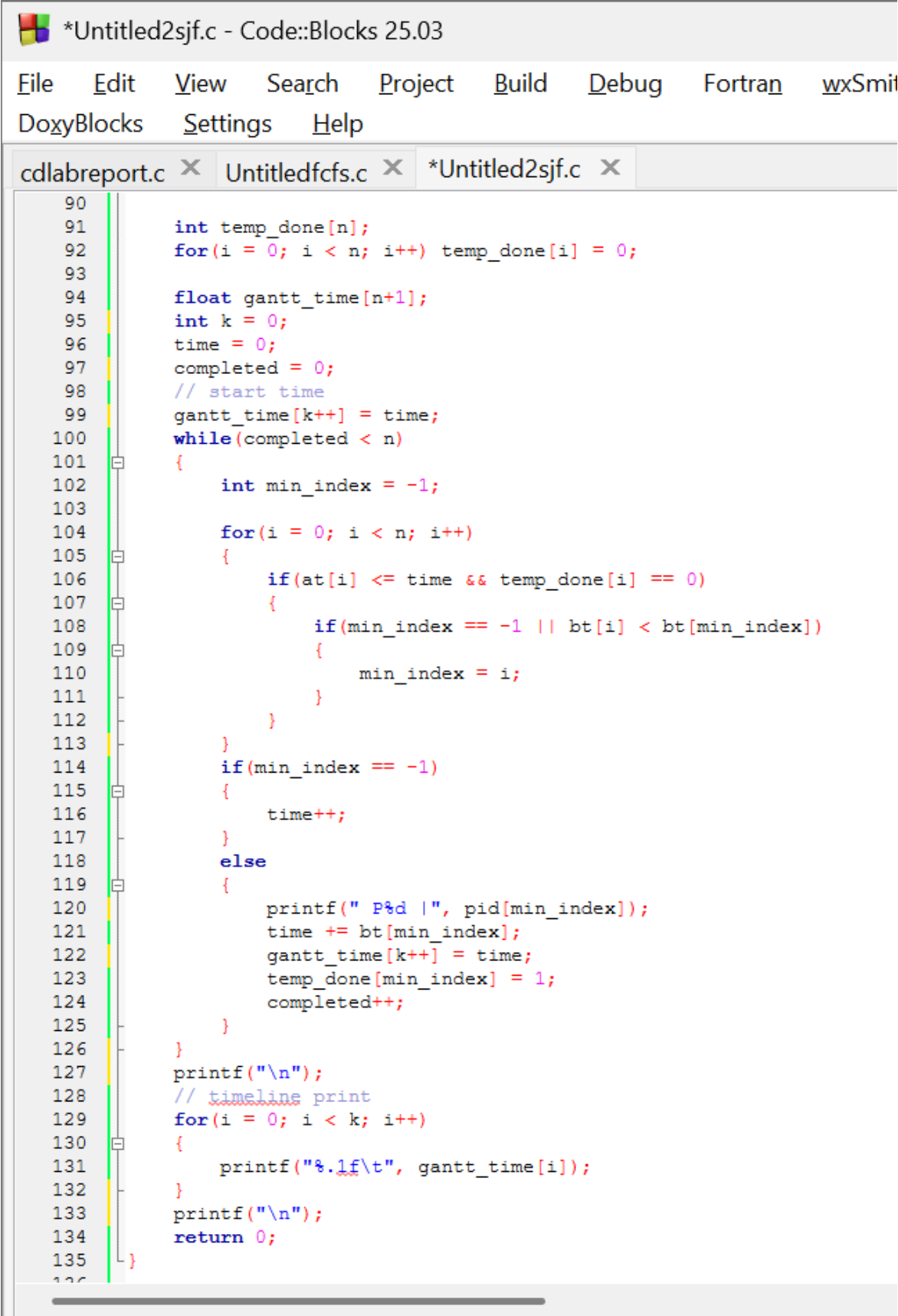
File Edit View Search Project Build Debug Fortran wxSn DoxyBlocks Settings Help

cdlabreport.c X Untitledfcfs.c X Untitled2sjf.c X

```

46         {
47             min_index = i;
48         }
49     }
50 }
51
52 // jodi kono process ready na thake
53 if(min_index == -1)
54 {
55     time++; // CPU idle
56 }
57 else
58 {
59     // process execute kortesi
60     ft[min_index] = time + bt[min_index];
61
62     tat[min_index] = ft[min_index] - at[min_index];
63     wt[min_index] = tat[min_index] - bt[min_index];
64
65     total_tat += tat[min_index];
66     total_wt += wt[min_index];
67
68     time = ft[min_index]; // time update
69
70     done[min_index] = 1;
71     completed++;
72 }
73 }
74
75 // Output table
76 printf("\n\nProcess\tAT\tBT\tFT\tTAT\tWT\n");
77
78 for(i = 0; i < n; i++)
79 {
80     printf("P%d\t%.1f\t%.1f\t%.1f\t%.1f\t%.1f\n",
81           pid[i], at[i], bt[i], ft[i], tat[i], wt[i]);
82 }
83
84 // Average
85 printf("\nAverage Turnaround Time = %.2f", total_tat / n);
86 printf("\nAverage Waiting Time = %.2f\n", total_wt / n);
87
88 // Gantt Chart generate kortesi
89 printf("\nGantt Chart:\n\n");
90
91 int temp_done[n];
92 for(i = 0; i < n; i++) temp_done[i] = 0;

```



```

90
91     int temp_done[n];
92     for(i = 0; i < n; i++) temp_done[i] = 0;
93
94     float gantt_time[n+1];
95     int k = 0;
96     time = 0;
97     completed = 0;
98     // start time
99     gantt_time[k++] = time;
100    while(completed < n)
101    {
102        int min_index = -1;
103
104        for(i = 0; i < n; i++)
105        {
106            if(at[i] <= time && temp_done[i] == 0)
107            {
108                if(min_index == -1 || bt[i] < bt[min_index])
109                {
110                    min_index = i;
111                }
112            }
113        }
114        if(min_index == -1)
115        {
116            time++;
117        }
118        else
119        {
120            printf(" P%d |", pid[min_index]);
121            time += bt[min_index];
122            gantt_time[k++] = time;
123            temp_done[min_index] = 1;
124            completed++;
125        }
126    }
127    printf("\n");
128    // timeline print
129    for(i = 0; i < k; i++)
130    {
131        printf("%.1f\t", gantt_time[i]);
132    }
133    printf("\n");
134    return 0;
135 }

```

Figure 1: Source Code in CodeBlocks

```

C:\Users\admin\Documents\L  ×  +  v  -  □  ×

Enter number of processes: 5

Process P1
Enter Arrival Time: 3
Enter Burst Time: 8

Process P2
Enter Arrival Time: 6
Enter Burst Time: 4

Process P3
Enter Arrival Time: 2
Enter Burst Time: 10

Process P4
Enter Arrival Time: 1
Enter Burst Time: 5

Process P5
Enter Arrival Time: 5
Enter Burst Time: 6

Process P5
Enter Arrival Time: 5
Enter Burst Time: 6

Process AT      BT      FT      TAT      WT
P1      3.0      8.0      24.0      21.0      13.0
P2      6.0      4.0      10.0      4.0       0.0
P3      2.0     10.0     34.0     32.0     22.0
P4      1.0      5.0      6.0      5.0       0.0
P5      5.0      6.0     16.0     11.0      5.0

Average Turnaround Time = 14.60
Average Waiting Time = 8.00

Gantt Chart:

| P4 | P2 | P5 | P1 | P3 |
0.0   6.0   10.0   16.0   24.0   34.0

Process returned 0 (0x0)  execution time : 30.839 s
Press any key to continue.

```

Figure 2: Program Execution Output

Discussion

Reflection

Through this experiment, I have learned how the SJF scheduling algorithm selects processes based on the shortest burst time. I have understood that this approach can significantly reduce the average waiting time compared to FCFS.

I have also learned how to dynamically select processes based on arrival time and manage execution flow efficiently.

Challenges Faced

Initially, I had difficulty selecting the shortest job among the available processes dynamically. I also ensured proper handling of CPU idle time when no process was ready. After resolving these logical issues, the program produced correct results.

Conclusion

In this experiment, I have successfully implemented the SJF (Non-Preemptive) scheduling algorithm using C. I have calculated all required scheduling parameters and visualized execution using a Gantt Chart.

This experiment has helped me understand how CPU scheduling can be optimized to improve system performance.
